

Daary Backend Full Project Report

This document is the full technical report for the backend system in this workspace. It is designed to include the complete project overview and the architecture details in the exported PDF. It covers how the system is built, what each file does, which functions exist, and how the business flows work end-to-end.

1. Project overview

This project is a Django REST API backend for a real-estate marketplace and developer dashboard. It supports:

- buyer authentication with OTP via Akedly
- property catalog browsing and search
- favorites management
- inquiry handling
- developer account and team management
- project management for developers
- marketing lead capture and export
- dashboard analytics
- AI-powered property recommendation survey

The backend is implemented as a set of modular Django apps under the `apps/` folder and mounted under the root Django project `daary_backend/`.

What the project makes

The project creates a backend platform that helps users:

- create a buyer account through OTP verification
- search, filter, and view real-estate listings
- save favorite properties
- ask inquiries about listings
- allow developers to manage their projects and account
- collect marketing leads and export them
- access dashboard analytics
- get AI recommendation results based on questionnaire answers

How the project works at a high level

A typical journey is: 1. User requests a challenge from the auth service. 2. User sends phone number and OTP challenge result. 3. The Akedly service sends or verifies a one-time password. 4. A JWT access token is created and returned. 5. The user browses properties, filters them, saves favorites, or submits inquiries. 6. The developer side manages accounts, team members, projects, dashboard stats, and lead exports. 7. The AI survey collects answers and matches products based on scoring logic.

2. Project structure

Core files:

- `manage.py`

- daary_backend/urls.py
- daary_backend/settings/base.py
- daary_backend/settings/dev.py
- confest.py

Application folders:

- apps/users
- apps/authentication
- apps/properties
- apps/search
- apps/favorites
- apps/inquiries
- apps/developer_accounts
- apps/projects
- apps/locations
- apps/marketing
- apps/dashboard
- apps/ai
- apps/common

3. Architecture and design patterns

The project follows a layered architecture:

1. Models: database schema and business entities 2. Selectors: read-only query helpers 3. Services: write logic, orchestration, validation logic 4. Serializers: request/response validation and serialization 5. Views: HTTP endpoints 6. URLs: URL route wiring 7. Tests: behavior validation 8. Tasks: background async jobs via Celery

This is a clean service-layer design for a Django app and matches common production patterns.

Why this architecture is strong

- each app is focused on one business domain
- read logic is separated from write logic
- views stay thin and mainly handle request/response flow
- repeated rules like validation, pagination, and permission logic live in shared modules
- data pipeline is easier to test and maintain

Full request flow example

A typical buyer flow starts like this: 1. The client requests GET /api/v1/properties/. 2. The PropertyListView receives the request. 3. A selector/filter function builds the queryset. 4. A serializer converts the DB rows to JSON. 5. The custom API renderer adds consistent metadata. 6. The response contains pagination data and property objects.

A typical OTP flow looks like this: 1. ChallengeView calls the Akedly client to get a challenge. 2. User solves the challenge and calls SendOtpView. 3. send_akedly_otp sends the OTP. 4. VerifyOtpView asks Akedly to validate the code. 5. The user record is activated and JWT tokens are generated. 6. The user is returned as authenticated with access and refresh tokens.

File-by-file function inventory

Root app

- manage.py — starts the Django project and loads settings.
- daary_backend/urls.py — maps top-level endpoints and includes app URLs.
- daary_backend/celery.py — configures Celery app and worker startup.
- daary_backend/settings/base.py — core project config.
- daary_backend/settings/dev.py — dev overrides.
- daary_backend/settings/prod.py — production config.
- daary_backend/settings/test.py — pytest-friendly config.

Common layer

- apps/common/models.py — TimestampedModel adds timestamps and ordering.
- apps/common/validators.py — Egyptian phone and birthday validation.
- apps/common/permissions.py — IsOwner permission.
- apps/common/pagination.py — standard pagination class.
- apps/common/renderers.py — custom response renderer.
- apps/common/exceptions.py — unified error handler.
- apps/common/akedly.py — AkedlyApiError, challenge, send OTP, verify OTP.

Users

- apps/users/models.py — custom User model and manager.
- apps/users/selectors.py — get_user_by_phone and get_user_by_id.
- apps/users/services.py — update_user_profile.
- apps/users/serializers.py — profile serializers.
- apps/users/views.py — authenticated profile endpoints.
- apps/users/urls.py — account routes.

Authentication

- apps/authentication/serializers.py — OTP request and verification serializers.
- apps/authentication/services.py — challenge, send and verify OTP logic.
- apps/authentication/views.py — challenge, send OTP, verify OTP, logout endpoints.
- apps/authentication/tasks.py — background auth tasks if used.
- apps/authentication/urls.py — route wiring.

Properties

- apps/properties/models.py — listing, agent, amenity, images, and enums.
- apps/properties/selectors.py — property fetch queries.
- apps/properties/services.py — increment_view_count.
- apps/properties/serializers.py — list/detail serializers.
- apps/properties/views.py — property listing and detail endpoints.
- apps/properties/tasks.py — property-related scheduled tasks like location medians.
- apps/properties/management/commands/seed_data.py — test data seeding command.

- apps/properties/management/commands/seed_properties.py — property data creation command.

Search

- apps/search/ranking.py — relevance score calculations.
- apps/search/services.py — search_properties filtering and ranking logic.
- apps/search/views.py — search API endpoint.
- apps/search/tasks.py — refresh price medians for search boosting.

Favorites

- apps/favorites/models.py — Favorite model and uniqueness constraint.
- apps/favorites/services.py — toggle_favorite.
- apps/favorites/serializers.py — favorite JSON representation.
- apps/favorites/views.py — list and toggle favorite endpoints.

Inquiries

- apps/inquiries/models.py — inquiry status and contact metadata.
- apps/inquiries/selectors.py — fetch inquiry lists/details.
- apps/inquiries/services.py — create_inquiry and update_lead.
- apps/inquiries/tasks.py — send webhook notifications.
- apps/inquiries/serializers.py — inquiry request/response schemas.
- apps/inquiries/views.py — inquiry CRUD/list endpoints.

Developer accounts

- apps/developer_accounts/models.py — account, permissions, team user models.
- apps/developer_accounts/selectors.py — team member lookup helpers.
- apps/developer_accounts/services.py — authentication, invites, updates, deactivation.
- apps/developer_accounts/permissions.py — developer access rules.
- apps/developer_accounts/authentication.py — custom developer auth token class.
- apps/developer_accounts/serializers.py — login, account, team serializers.
- apps/developer_accounts/views.py — account and team management APIs.

Projects

- apps/projects/models.py — project and project image data.
- apps/projects/selectors.py — account-scoped project queries.
- apps/projects/services.py — create, update, delete project operations.
- apps/projects/serializers.py — project serializers.
- apps/projects/views.py — list/create/detail/stats endpoints.

Locations

- apps/locations/models.py — hierarchical location data.
- apps/locations/serializers.py — location response schema.
- apps/locations/views.py — location listing endpoint.

Marketing

- apps/marketing/models.py — MarketingLead and sources.

- apps/marketing/serializers.py — lead create serializer.
- apps/marketing/views.py — create lead, list leads, export leads.

Dashboard

- apps/dashboard/services.py — get_dashboard_stats for account metrics.
- apps/dashboard/views.py — overview endpoint.
- apps/dashboard/urls.py — dashboard route mapping.

AI

- apps/ai/consultation_scoring.py — scoring functions: homebuyer/investor/property track.
- apps/ai/models.py — survey session/question/answer/result models.
- apps/ai/services.py — survey session state and match generation.
- apps/ai/serializers.py — survey result and answer serializers.
- apps/ai/views.py — survey start/answer/results endpoints.

Tests

- apps/*/tests/*.py — coverage of auth, models, views, services, and AI flows.

4. Root project configuration

manage.py

Purpose:

- Django entry point
- sets DJANGO_SETTINGS_MODULE
- calls execute_from_command_line()

daary_backend/urls.py

Routes:

- health endpoint: /health/
- admin: /admin/
- API v1: /api/v1/
- API docs: /api/docs/

The root includes this API structure:

- auth
- account
- properties
- favorites
- inquiries
- ai
- search
- locations
- contact

- developer

daary_backend/settings/base.py

This is the main settings file and contains:

- installed apps
- custom user model
- middleware stack
- database config
- DRF config
- JWT config
- CORS config
- logging config
- Celery settings
- OpenAPI config
- SMS config
- Akedly config
- AI provider config

Important settings:

- JWT auth - custom renderer - standard pagination - exception handler - throttling

- AUTH_USER_MODEL = "users.User"
- REST_FRAMEWORK includes:
- JWT lifetime defaults are 15 minutes access and 7 days refresh
- PAGE_SIZE = 20

daary_backend/settings/dev.py

Development overrides:

- DEBUG = True
- SQLite default database
- CORS relaxed for local dev
- console email backend
- token lifetime extended to 60 minutes

5. Shared common utilities

apps/common/models.py

TimestampedModel

- abstract model
- adds created_at and updated_at
- default ordering by newest first

apps/common/exceptions.py

custom_exception_handler()

- success - data - message - errors - meta

- wraps DRF response errors in a consistent schema:

apps/common/pagination.py

StandardPagination

- default page size 20
- page_size_query_param = "page_size"
- max page size 100
- paginated response metadata includes count, page, pages, next, previous

apps/common/permissions.py

IsOwner

- checks whether request user owns object
- supports obj.user or obj.user_id

apps/common/validators.py

- allows valid Egyptian mobile formats - normalizes +2010... to 010... - rejects invalid numbers

- rejects future dates - enforces age ≥ 18 - rejects unrealistic ages > 120

- validate_egyptian_phone(value)
- validate_birthday(value)

apps/common/akedly.py

This is the external OTP provider integration layer.

Main entities:

- AkedlyApiError
- _get_credentials()
- _parse_akedly_response()
- get_challenge()
- send_otp()
- verify_otp()

What it does:

- reads Akedly credentials from Django settings
- requests a challenge for proof-of-work
- sends OTP to verification address
- verifies OTP codes
- handles retry logic and error normalization

6. Buyer users and profile management

apps/users/models.py

UserManager

- creates user with validated phone
- sets password
- validates values with full_clean

User

- buyer

- custom auth model
- uses phone as username
- no email field in buyer model
- role values:

Fields:

- phone
- name
- birthday
- role
- is_active
- is_staff
- timestamps

Important logic:

- is_active default is False until OTP verification completes
- USER_NAME_FIELD = "phone"

apps/users/selectors.py

- get_user_by_phone(phone)
- get_user_by_id(user_id)

apps/users/services.py

update_user_profile(user, **fields)

- allows update of name and birthday
- phone is immutable
- validates before save

apps/users/serializers.py

- UserProfileSerializer
- UserUpdateSerializer

apps/users/views.py

MeView

- GET /api/v1/account/me/
- PATCH /api/v1/account/me/

apps/users/urls.py

- me/

7. OTP authentication flow (buyer)

apps/authentication/serializers.py

- PowSolutionSerializer
- SendOtpSerializer
- VerifyOtpSerializer
- AuthTokenSerializer

apps/authentication/services.py

Main functions:

- get_akedly_challenge()
- send_akedly_otp(...)
- verify_akedly_otp_and_authenticate(...)

Behavior:

- converts local Egyptian phone number into Akedly-friendly format
- gets or creates a User
- sets user active
- creates SimpleJWT access and refresh tokens
- returns is_new_user

apps/authentication/views.py

- OtpRateThrottle
- _handle_akedly_error(err)
- ChallengeView
- SendOtpView
- VerifyOtpView
- LogoutView

API routes in apps/authentication/urls.py

- challenge/
- send-otp/
- verify-otp/
- token/refresh/
- logout/

Endpoints

1) Challenge GET /api/v1/auth/challenge/ Purpose:

- retrieve PoW challenge and Turnstile config from Akedly

2) Send OTP POST /api/v1/auth/send-otp/ Body:

```
{ "phone_number": "01012345678", "pow_solution": { "challengeToken": "tok-xyz", "nonce": 42 }, "turnstile_token": "optional", "digits": 4 }
```

3) Verify OTP POST /api/v1/auth/verify-otp/ Body:

```
{ "transaction_req_id": "req-001", "otp": "1234", "phone_number": "01012345678" }
```

Returns JWT access token and refresh token plus serialised user.

4) Refresh token POST /api/v1/auth/token/refresh/ This is provided by rest_framework_simplejwt and is already mounted via the URL config.

5) Logout POST /api/v1/auth/logout/ Requires auth and blacklists supplied refresh token.

8. Properties catalog

apps/properties/models.py

Main entities:

- PropertyType
- CompletionStatus
- Furnishing
- PaymentMethod
- Agent
- Amenity
- Property
- PropertyImage

At a high level:

- property is a listing record
- agent is assigned to a property
- amenities are many-to-many
- each property can have multiple images

apps/properties/selectors.py

- get_property_by_id(pk)
- get_properties_queryset(filters)

These helpers fetch property data with optimized queries.

apps/properties/serializers.py

- AgentSerializer
- PropertyImageSerializer
- PropertyListSerializer
- PropertyDetailSerializer

apps/properties/services.py

increment_view_count(property, request)

- prevents duplicate count inflation using cache
- with same IP/session/user within 5 minutes it will not count again

apps/properties/views.py

- PropertyListView
- PropertyDetailView

Endpoints

- GET /api/v1/properties/
- GET /api/v1/properties/

Example query params

- location
- property_type
- completion_status
- furnishing
- payment_method
- is_featured
- ordering

9. Search and ranking engine

apps/search/ranking.py

Purpose:

- calculates relevance score for properties
- supports deterministic ordering

Key functions:

- `_get_segment_median_price(location, property_type)`
- `compute_relevance_score(...)`
- `apply_ordering(...)`

Scoring weights:

- recency
- completeness
- price competitiveness
- filter match

apps/search/services.py

Main function:

- `search_properties(filters=None, ordering=None)`

Supported filters:

- location
- propertyTypes
- bedrooms
- bathrooms
- minPrice
- maxPrice
- minArea

- maxArea
- completionStatus
- furnishing
- amenities
- payment

This file includes Arabic alias maps for search values such as:

- 
- 
- 
- 
- 
- 

apps/search/views.py

PropertySearchView

- GET /api/v1/search/

This accepts all request params as filters and applies pagination.

10. Favorites system

apps/favorites/models.py

Favorite

- link between authenticated user and a property
- uniqueness enforced on (user, property)

apps/favorites/services.py

toggle_favorite(user, property)

- add or remove favorite
- returns action and is_favorited

apps/favorites/views.py

- FavoriteListView
- FavoriteToggleView

Endpoints

- GET /api/v1/favorites/
- POST /api/v1/favorites//toggle/

11. Inquiries and buyer lead handling

apps/inquiries/models.py

Model Inquiry stores:

- property
- user
- name
- phone
- message
- inquiry type
- status
- preferred contact method
- preferred time
- rating
- source
- requirement summary
- agent_notified

apps/inquiries/selectors.py

- get_user_inquiries(user)
- get_inquiry_by_id(pk, user=None)

apps/inquiries/services.py

- create_inquiry(data, user=None, client_ip=None)
- update_lead(inquiry, status=None, rating=None)

Important logic:

- validates phone using validate_egyptian_phone
- verifies property exists
- prevents duplicate spam submissions with cache cooldown
- triggers async notification task

apps/inquiries/tasks.py

send_inquiry_notification(self, inquiry_id)

- reads inquiry and related property/agent
- builds message text
- posts to configured webhook URL if one is set
- marks inquiry as notified

apps/inquiries/views.py

- InquiryCreateView
- MyInquiriesListView
- InquiryDetailView

Endpoints

- POST /api/v1/inquiries/
- GET /api/v1/inquiries/my/
- GET /api/v1/inquiries//

12. Developer accounts and team RBAC

apps/developer_accounts/models.py

Main entities:

- Permission
- DeveloperAccount
- DeveloperUser

Developer user model:

- email/password authentication
- belongs to an account
- has permissions many-to-many
- is_primary marks the account owner
- status is either active or inactive

apps/developer_accounts/authentication.py

DeveloperTokenAuthentication

- resolves DRF token to DeveloperUser

apps/developer_accounts/permissions.py

- IsDeveloperAuthenticated
- HasDeveloperPermission

apps/developer_accounts/services.py

- authenticate_developer(email, password)
- update_developer_account(user, data)
- invite_team_member(account, data)
- update_team_member(member, data)
- deactivate_team_member(member)
- delete_developer_account(user)

apps/developer_accounts/views.py

- LoginView
- RegisterView
- AccountView
- TeamListView
- TeamCreateView
- TeamDetailView

Routes

- POST /api/v1/developer/login/
- POST /api/v1/developer/register/
- GET /api/v1/developer/account/

- PUT /api/v1/developer/account/
- DELETE /api/v1/developer/account/
- GET /api/v1/developer/team/
- POST /api/v1/developer/team/invite/
- PUT /api/v1/developer/team//
- DELETE /api/v1/developer/team//

13. Developer projects management

apps/projects/models.py

- ProjectType
- ProjectStatus
- Project
- ProjectImage

The project is linked to a DeveloperAccount and a Location.

apps/projects/services.py

- create_project(account, data, files)
- update_project(project, data, files)
- delete_project(project)

apps/projects/views.py

- ProjectListCreateView
- ProjectDetailView
- ProjectStatsView

Endpoints

- GET /api/v1/developer/projects/
- POST /api/v1/developer/projects/
- GET /api/v1/developer/projects//
- PUT /api/v1/developer/projects//
- DELETE /api/v1/developer/projects//
- GET /api/v1/developer/projects//stats/

These support file upload and project CRUD.

14. Locations hierarchy

apps/locations/models.py

Location

- has name
- optional parent foreign key
- supports governorate/area/compound-style structure

apps/locations/views.py

LocationListView

- GET /api/v1/locations/
- supports search and parent query filters

15. Marketing leads

apps/marketing/models.py

MarketingLead

- captures campaign leads
- has source enum and optional project relation

apps/marketing/views.py

- MarketingLeadCreateView
- DeveloperLeadListView
- DeveloperLeadExportView

Endpoints

- POST /api/v1/contact/lead/
- GET /api/v1/developer/leads/
- GET /api/v1/developer/leads/export/

The export writes CSV with UTF-8 BOM for Arabic compatibility in Excel.

16. Dashboard analytics

apps/dashboard/services.py

get_dashboard_stats(account) Returns:

- projects total + active
- top projects by leads
- leads total + last 30 days
- leads by source
- location medians

apps/dashboard/views.py

DashboardOverviewView

- GET /api/v1/developer/dashboard/overview/

17. AI recommendation survey

apps/ai/models.py

- SurveySession
- SurveyQuestion

- SurveyAnswer
- SurveyResult

apps/ai/consultation_scoring.py

Two scoring flows:

- score_homebuyer_property(...)
- score_investor_property(...)
- score_property_for_track(...)

This calculates a 0-100 match score based on budget, property type, location, timeline, and amenities.

apps/ai/services.py

- _ensure_question_bank()
- start_survey(user)
- _get_session_for_request(request, session_id)
- _serialize_matches(session)
- answer_survey_question(request, data)

apps/ai/views.py

- SurveyStartView
- SurveyAnswerView
- SurveyResultsView

Routes

- POST /api/v1/ai/survey/start/
- POST /api/v1/ai/survey/answer/
- GET /api/v1/ai/survey//results/

This survey asks a fixed question sequence and returns the best matching properties.

18. Tests coverage summary

Authentication tests

- apps/authentication/tests/test_akedly_client.py
- apps/authentication/tests/test_views.py

Covers:

- challenge success/failure
- OTP send validation
- OTP verify flows
- JWT refresh and logout
- Akedly retry and error handling

User tests

- apps/users/tests/test_models.py
- apps/users/tests/test_views.py

Covers:

- phone validation
- underage validation
- profile fetch and update
- immutable phone behavior

Property tests

- apps/properties/tests/test_models.py
- apps/properties/tests/test_views.py

Covers:

- property creation
- image logic
- view count behavior
- filtering and API output

Search tests

- apps/search/tests/test_services.py

Covers:

- filter aliases
- bedroom and bathroom filtering
- area and price filter logic
- relevance ordering determinism

Favorites tests

- apps/favorites/tests/test_views.py

Covers:

- auth checks
- toggle behavior
- user-specific favorites isolation

Inquiries tests

- apps/inquiries/tests/test_views.py

Covers:

- guest inquiry creation
- authenticated inquiry creation
- duplicate anti-spam cooldown
- invalid phone rejection
- user inquiry list

AI survey test

- apps/ai/tests/test_survey.py

Covers:

- survey start

- answer progression
- result generation

19. Important environment and deployment constants

Settings include:

- JWT config
- CORS configuration
- Celery broker and result backend
- SMS backend
- Akedly credentials
- AI provider config

The actual storage engine defaults to SQLite in development and uses env-driven config in production.

20. API quick reference

Buyer APIs

- GET /health/
- GET /api/v1/auth/challenge/
- POST /api/v1/auth/send-otp/
- POST /api/v1/auth/verify-otp/
- POST /api/v1/auth/token/refresh/
- POST /api/v1/auth/logout/
- GET /api/v1/account/me/
- PATCH /api/v1/account/me/
- GET /api/v1/properties/
- GET /api/v1/properties//
- GET /api/v1/search/
- GET /api/v1/favorites/
- POST /api/v1/favorites//toggle/
- POST /api/v1/inquiries/
- GET /api/v1/inquiries/my/
- GET /api/v1/inquiries//
- POST /api/v1/contact/lead/
- POST /api/v1/ai/survey/start/
- POST /api/v1/ai/survey/answer/
- GET /api/v1/ai/survey//results/

Developer APIs

- POST /api/v1/developer/login/

- POST /api/v1/developer/register/
- GET /api/v1/developer/account/
- PUT /api/v1/developer/account/
- DELETE /api/v1/developer/account/
- GET /api/v1/developer/team/
- POST /api/v1/developer/team/invite/
- PUT /api/v1/developer/team//
- DELETE /api/v1/developer/team//
- GET /api/v1/developer/projects/
- POST /api/v1/developer/projects/
- GET /api/v1/developer/projects//
- PUT /api/v1/developer/projects//
- DELETE /api/v1/developer/projects//
- GET /api/v1/developer/projects//stats/
- GET /api/v1/developer/dashboard/overview/
- GET /api/v1/developer/leads/
- GET /api/v1/developer/leads/export/

System APIs

- GET /api/v1/locations/
- GET /api/docs/schema/
- GET /api/docs/swagger-ui/
- GET /api/docs/redoc/

21. Summary of project purpose

This backend is a full real-estate platform backend that supports:

- property marketplace discovery
- OTP-based buyer authentication
- inquiry submission and lead management
- developer account and project operations
- marketing lead collection and analytics
- AI recommendation flow for buyer matching

The codebase is structured clearly, uses service/query patterns effectively, and is strong in modularity and test coverage.

22. Final review

Strengths

- clear app boundaries
- service-layer separation

- consistent API response format
- good validation discipline
- proper feature grouping
- strong test coverage in key flows

Improvement opportunities

- some project update logic could be more explicit around gallery replacement behavior
- AI scoring is more homebuyer-oriented in some sections and can be widened for investor logic
- a few areas could standardize permission flows even more
- production-level media validation and task operationalization could be enhanced

23. How to run

From the project root:

```
python manage.py migrate python manage.py runserver
```

For test runner:

```
pytest
```

For seed data:

```
python manage.py seed_data
```

This document summarizes the current backend codebase in the workspace and captures the main architecture, app structure, endpoints, and business flow.